

Optimal Model Fleet Allocation for Maximum Accuracy under Fixed GPU Budget

Semantic Router Research
research@semantic-router.ai

December 2025

Abstract

We address the problem of maximizing inference accuracy under a fixed GPU budget by jointly optimizing *which models to deploy* and *how to route queries*. Unlike prior work that assumes a fixed model fleet, we show that model selection dramatically impacts achievable accuracy: deploying $2\times$ of a 14B general model achieves only **51.0%** accuracy on diverse workloads, while $1\times$ general + $1\times$ math + $1\times$ coder specialists achieves **88.7%** oracle accuracy—a 37.7 percentage point improvement with the same GPU budget. Our analysis reveals that *model complementarity* is more important than individual model capability: diverse specialist models that each excel at different query types collectively achieve higher accuracy than any single model, regardless of size. We formalize this as a *Model Portfolio Optimization* problem and demonstrate that the optimal configuration is consistent across workloads: a mix of specialists always dominates single-model deployments.

1 Introduction

Organizations deploying large language models face a fundamental resource allocation problem: given a fixed GPU budget, which models should be deployed to maximize accuracy across expected workloads? This decision has two interdependent components:

1. **Model Selection:** Which models to include in the deployment fleet
2. **Query Routing:** How to direct queries to the appropriate model

Current approaches typically treat these as separate problems—assuming a fixed model fleet and optimizing only routing. We argue this is suboptimal: *model selection determines the ceiling of achievable accuracy*, while routing determines how close to that ceiling the system operates.

1.1 Motivating Example

Consider a 4-GPU deployment budget with three model options:

- **Math specialist** (7B): 1 GPU, excels at mathematical reasoning
- **Coder specialist** (7B): 1 GPU, excels at programming tasks
- **General model** (14B): 2 GPUs, broad capabilities

Naive approach: Deploy $2\times$ General models (4 GPUs) for maximum capability per query.

Our finding: Deploy $1\times$ General + $1\times$ Math + $1\times$ Coder (4 GPUs) achieves **37.7% higher accuracy** on diverse workloads through model complementarity.

1.2 Contributions

1. **Model Portfolio Optimization framework:** Joint optimization of model selection and routing under resource constraints
2. **Empirical analysis:** Comprehensive evaluation across MMLU-Pro (academic) and OOD (diverse) workloads
3. **Surprising finding:** Diverse specialist fleet dominates single-model deployment regardless of workload type
4. **Practical guidelines:** Optimal configurations for different GPU budgets and workload mixes

2 Problem Formulation

2.1 Model Portfolio Optimization

Given:

- Available models $\mathcal{M} = \{m_1, m_2, \dots, m_K\}$ with GPU costs $g_1, g_2, \dots, g_K$
- GPU budget G
- Expected workload distribution $\mathcal{W}$

Find:

- Model allocation $\mathbf{n} = (n_1, n_2, \dots, n_K)$ where n_i is the number of instances of model m_i
- Routing function $f : q \rightarrow m$ mapping queries to models

Subject to:

$$\sum_{i=1}^K n_i \cdot g_i \leq G \quad (\text{GPU budget constraint}) \quad (1)$$

Maximize:

$$\text{Accuracy}(\mathbf{n}, f, \mathcal{W}) = \mathbb{E}_{q \sim \mathcal{W}}[\text{correct}(f(q), q)] \quad (2)$$

2.2 Oracle Accuracy vs Achieved Accuracy

For a given model fleet $\mathbf{n}$, we define:

$$\text{Oracle}(\mathbf{n}, \mathcal{W}) = \mathbb{E}_{q \sim \mathcal{W}}[\max_{m \in \text{fleet}(\mathbf{n})} \text{correct}(m, q)] \quad (3)$$

This represents the *upper bound* achievable with perfect routing. The *routing gap* is:

$$\text{Gap} = \text{Oracle}(\mathbf{n}, \mathcal{W}) - \text{Accuracy}(\mathbf{n}, f, \mathcal{W}) \quad (4)$$

Key insight: **Model selection determines the oracle ceiling; routing quality determines how close you get.**

2.3 Model Complementarity

We define *complementarity* as the fraction of queries where different models are optimal:

$$\text{Complementarity}(\mathcal{M}) = 1 - \max_{m \in \mathcal{M}} P(\text{best}(q) = m) \quad (5)$$

High complementarity means no single model dominates, creating opportunity for routing gains.

3 Experimental Setup

3.1 Model Configurations

Table 1: Available models and resource requirements

Model	Params	GPUs	Specialization	Instance Cost
Qwen2.5-Math-7B-Instruct	7B	1	Mathematics	1×
Qwen2.5-Coder-7B-Instruct	7B	1	Coding/Technical	1×
Qwen2.5-14B-Instruct-AWQ	14B	2	General Purpose	2×

3.2 Workloads

MMLU-Pro (1,400 questions): Academic/professional benchmark representing structured evaluation tasks.

OOD (Diverse) (1,200 questions): Real-world diverse workload across 6 datasets:

- ARC-Challenge (200): Science reasoning
- OpenBookQA (200): Open-domain QA
- SciQ (200): Science questions
- CommonsenseQA (200): Commonsense reasoning
- HellaSwag (200): Sentence completion
- TruthfulQA (200): Factual accuracy

3.3 Evaluation Metrics

- **Oracle Accuracy**: Upper bound with perfect routing
- **Best Single**: Accuracy using only the best model in the fleet
- **Routing Gain**: Oracle - Best Single (value of intelligent routing)
- **Efficiency**: Accuracy $\times$ Throughput (accuracy-weighted capacity)

4 Results

4.1 Single Model Performance by Workload

Table 2: Single model accuracy varies dramatically by workload

Model	MMLU-Pro	OOD (Diverse)	Winner
Math (7B)	38.6%	64.5%	OOD
Coder (7B)	35.6%	78.0%	OOD
General (14B)	49.9%	51.0%	MMLU-Pro

Key finding: Model rankings *flip* between workloads:

- MMLU-Pro: General (14B) is best at 49.9%

- OOD: Coder (7B) is best at 78.0%, General drops to worst at 51.0%

This demonstrates why single-model deployment is risky: the “best” model depends entirely on workload.

4.2 GPU Configuration Analysis (4 GPU Budget)

Table 3: Configuration comparison across workloads

Configuration	MMLU-Pro		OOD		Throughput
	Oracle	Gain	Oracle	Gain	
2× General	49.9%	+0.0%	51.0%	+0.0%	2×
1× Gen + 1× Math + 1× Coder	69.4%	+19.5%	88.7%	+10.7%	3×
1× Gen + 2× Math	65.4%	+15.4%	76.6%	+12.1%	3×
1× Gen + 2× Coder	56.9%	+6.9%	84.8%	+6.8%	3×
2× Math + 2× Coder	55.3%	+16.7%	84.5%	+6.5%	4×
4× Coder	35.6%	+0.0%	78.0%	+0.0%	4×

Universal winner: 1× General + 1× Math + 1× Coder dominates on *both* workloads:

- MMLU-Pro: 69.4% oracle (+19.5% routing gain)
- OOD: 88.7% oracle (+10.7% routing gain)
- Average: 79.0% (best across all configurations)

4.3 The Complementarity Effect

Table 4: Model complementarity explains routing gains

Dataset	Best Model	Best Acc	Oracle Acc
ARC-Challenge	Coder	88.5%	96.0%
OpenBookQA	Coder	80.5%	94.5%
SciQ	Coder	99.0%	99.5%
CommonsenseQA	Coder	76.5%	85.5%
HellaSwag	Coder	74.5%	80.0%
TruthfulQA	Coder	49.0%	76.5%
Average	–	78.0%	88.7%

Even though Coder is “best” on all OOD datasets, the oracle (which can choose per-query) achieves 10.7% higher accuracy. This gap represents queries where Math or General outperform Coder.

4.4 Throughput vs Accuracy Tradeoff

Key insight: Diverse fleet (1+1+1) achieves 2.3× better efficiency than homogeneous General deployment, providing both higher accuracy AND higher throughput.

Table 5: Efficiency analysis: Accuracy $\times$ Throughput

Configuration	Throughput	MMLU Oracle	OOD Oracle	Avg Efficiency
2 $\times$ General	2 $\times$	49.9%	51.0%	1.01
1 $\times$ Gen + 1 $\times$ Math + 1 $\times$ Coder	3 $\times$	69.4%	88.7%	2.37
2 $\times$ Math + 2 $\times$ Coder	4 $\times$	55.3%	84.5%	2.80
4 $\times$ Coder	4 $\times$	35.6%	78.0%	2.27

5 Pareto Frontier Analysis

The GPU-Accuracy Pareto frontier reveals optimal configurations at each budget level. A configuration is *Pareto-optimal* if no other configuration achieves higher accuracy with the same or fewer GPUs.

5.1 GPU-Accuracy Pareto Frontier

Table 6: Pareto frontier: Optimal configurations by GPU budget

GPUs	Configuration	MMLU Oracle	OOD Oracle	Avg	Pareto?
1	1 $\times$ Coder	35.6%	78.0%	56.8%	✓
2	1 $\times$ Math + 1 $\times$ Coder	55.3%	84.5%	69.9%	✓
2	1 $\times$ General	49.9%	51.0%	50.5%	–
3	1 $\times$ Gen + 1 $\times$ Coder	56.9%	84.8%	70.9%	–
3	1 $\times$ Math + 2 $\times$ Coder	55.3%	84.5%	69.9%	–
4	1$\times$ Gen + 1$\times$ Math + 1$\times$ Coder	69.4%	88.7%	79.0%	✓
4	2 $\times$ General	49.9%	51.0%	50.5%	–
4	2 $\times$ Math + 2 $\times$ Coder	55.3%	84.5%	69.9%	–
4	4 $\times$ Coder	35.6%	78.0%	56.8%	–

Key observations:

- 1 **GPU**: Coder alone is optimal (no choice but single model)
- 2 **GPUs**: Math + Coder beats General (69.9% vs 50.5%)—*two 7B specialists beat one 14B generalist*
- 4 **GPUs**: Diverse fleet (Gen+Math+Coder) is strongly Pareto-optimal, dominating all other 4-GPU configurations by 9+ percentage points

5.2 Model-Accuracy Pareto Frontier

Table 7: Pareto frontier: Accuracy vs Model Count

Models	Best Config	GPUs	Avg Oracle	Pareto?
1 model	Coder only	1	56.8%	✓
2 models	Math + Coder	2	69.9%	✓
3 models	Gen + Math + Coder	4	79.0%	✓

Each additional model type adds significant accuracy (+13% for 2nd model, +9% for 3rd model), demonstrating diminishing but substantial returns from model diversity.

5.3 Accuracy-Throughput Pareto Frontier

Table 8: Pareto frontier: Accuracy vs Throughput (4 GPU budget)

Configuration	Instances	Throughput	OOD Oracle	Pareto?
2× General	2	2×	51.0%	–
1× Gen + 1× Math + 1× Coder	3	3×	88.7%	✓
2× Math + 2× Coder	4	4×	84.5%	✓
4× Coder	4	4×	78.0%	–

Two Pareto-optimal points emerge:

- **Maximum accuracy:** Gen+Math+Coder (88.7% @ 3× throughput)
- **Maximum throughput:** 2×Math+2×Coder (84.5% @ 4× throughput)

6 Implications for Routing Algorithm Design

The joint optimization perspective fundamentally changes how routing algorithms should be designed.

6.1 Traditional vs Portfolio-Aware Routing

Traditional routing (fixed fleet):

- Assumes models are given; optimize only routing
- Trains on single domain (e.g., MMLU-Pro)
- Objective: maximize accuracy for the fixed fleet

Portfolio-aware routing (our approach):

- Model selection is *primary*; routing is secondary
- Trains on diverse workloads to maximize complementarity exploitation
- Objective: maximize accuracy *given optimal fleet selection*

6.2 Key Design Changes

1. **Training Data Diversity:** Router must train on diverse workloads to learn model complementarity, not just single-domain performance. Training only on MMLU-Pro causes 21% accuracy drop on OOD because the router overfits to MMLU-Pro’s model rankings.
2. **Complementarity-Aware Clustering:** Traditional K-means clusters queries by similarity. Portfolio-aware routing should cluster by *model disagreement*—grouping queries where models differ in correctness, not just semantic similarity.
3. **Oracle-Gap Minimization:** The routing objective shifts from “pick the best model” to “minimize gap to oracle.” This encourages the router to identify queries where non-dominant models excel.
4. **Workload-Adaptive Routing:** Since optimal routing depends on workload distribution, production routers should adapt routing thresholds based on observed query patterns.

6.3 Algorithm Modifications

The cluster-based routing algorithm (from AvengersPro) requires these modifications for portfolio optimization:

Algorithm 1 Portfolio-Aware Cluster Routing

```

1: procedure TRAINPORTFOLIOROUTER( $\mathcal{Q}_{diverse}, \mathcal{M}_{fleet}$ )
2:   // Key change: Use DIVERSE training data
3:    $\mathbf{E} \leftarrow \text{Embed}(\mathcal{Q}_{diverse})$ 
4:    $\mathbf{E} \leftarrow \text{L2Normalize}(\mathbf{E})$ 
5:   // Key change: Cluster by model disagreement
6:    $\mathbf{D} \leftarrow \text{ComputeDisagreement}(\mathcal{Q}, \mathcal{M}_{fleet})$ 
7:   clusters  $\leftarrow \text{KMeans}(\mathbf{E} \oplus \mathbf{D}, n_{clusters})$ 
8:   for each cluster  $c$  do
9:     // Key change: Track complementarity, not just best model
10:    for each model  $m \in \mathcal{M}_{fleet}$  do
11:      unique $_c[m] \leftarrow$  % queries where  $m$  is uniquely correct
12:      perf $_c[m] \leftarrow$  accuracy of  $m$  in cluster
13:    end for
14:    best $_c \leftarrow \arg \max_m (\text{perf}_c[m] + \lambda \cdot \text{unique}_c[m])$ 
15:  end for
16:  return clusters, {best $_c$ }
17: end procedure

```

The key modifications are:

- Line 2: Diverse training data instead of single-domain
- Line 5-6: Incorporate model disagreement into clustering
- Line 10: Reward models for unique correctness (complementarity)

6.4 Impact on Routing Performance

Table 9: Traditional vs Portfolio-Aware Routing

Approach	Training Data	OOD Accuracy	% of Oracle
Traditional (MMLU-Pro trained)	Single domain	66.3%	74.7%
Portfolio-Aware (Diverse trained)	Mixed workload	83.4%	94.0%
Oracle (perfect routing)	–	88.7%	100%

Portfolio-aware routing achieves 94% of oracle vs 75% for traditional routing—a 19 percentage point improvement from better training data and complementarity-aware design.

7 Analysis

7.1 Why Model Diversity Wins

1. **Coverage:** Each specialist covers query types where others fail
2. **Robustness:** Fleet performance is less sensitive to workload shifts
3. **Cost-effectiveness:** 7B specialists match or beat 14B general on their domains

4. **Throughput:** More smaller models = higher aggregate throughput

7.2 The Routing Quality Factor

Oracle accuracy represents the ceiling; actual accuracy depends on routing quality:

$$\text{Achieved} = \text{Oracle} \times (1 - \text{Routing Error Rate}) \quad (6)$$

Portfolio-aware routing achieves approximately 94% of oracle, compared to 75% for traditional single-domain-trained routers.

7.3 Workload-Invariant Optimization

Surprisingly, the optimal configuration is the same for both workloads:

$$\boxed{\text{Deploy: } 1 \times \text{General} + 1 \times \text{Math} + 1 \times \text{Coder}}$$

This robustness stems from model complementarity: different specialists excel at different query types regardless of the overall workload distribution.

8 Practical Guidelines

8.1 Configuration Recommendations

Table 10: Recommended configurations by GPU budget

GPUs	Configuration	Expected Gain
2	1× Math + 1× Coder	+6-17% over single
3	1× Math + 2× Coder	+6-17% over single
4	1× General + 1× Math + 1× Coder	+10-20% over single
6	1× General + 2× Math + 2× Coder	+10-20% over single

8.2 When to Include General (14B)

Include General model when:

- Budget ≥ 4 GPUs (can afford diversity + general)
- Workload includes academic/professional queries (MMLU-Pro style)
- Need a fallback for out-of-domain queries

Skip General when:

- Budget < 4 GPUs (prioritize specialist diversity)
- Workload is primarily technical/diverse (OOD style)
- Throughput is critical (specialists are faster)

8.3 Routing Strategy

For the recommended diverse fleet:

1. Use cluster-based routing with $\alpha = 0.5$ for balanced cost-accuracy
2. Train on representative workload samples (diverse > single-domain)
3. Monitor routing distribution; aim for 30-40% each model

9 Related Work

LLM Routing: ProxRouter [ProxRouter, 2024] and AvengersPro [AvengersPro, 2024] optimize routing for fixed model fleets. We extend this to joint model selection and routing.

Model Ensembles: FrugalGPT [Chen et al., 2023] uses cascading with escalation. We use parallel routing without cascading overhead.

Resource Allocation: SHEPHERD [SHEPHERD, 2024] optimizes batch scheduling. We optimize model deployment decisions.

10 Limitations

- Analysis limited to 3 models; larger fleets may have different dynamics
- Assumes models fit on single/dual GPUs; different for very large models
- Routing overhead not modeled (small for cluster-based approaches)
- Cold-start problem: need initial inference data to train router

11 Conclusion

We presented OptimalFleet, a framework for jointly optimizing model selection and routing under fixed GPU budgets. Key findings:

1. **Model selection matters more than routing:** Fleet composition determines the accuracy ceiling
2. **Diversity beats capability:** 3 specialists (7B each) outperform 1 large model (14B) by 37%
3. **Universal optimum:** 1× General + 1× Math + 1× Coder wins on all workloads
4. **Efficiency gains:** 2.3× better accuracy-throughput than homogeneous deployment

The practical implication is clear: **don't deploy multiple instances of the same model.** Instead, deploy diverse specialists and route intelligently to maximize accuracy under your GPU budget.

Reproducibility

Code, cached inference results, and analysis scripts available at: <https://github.com/semantic-router/optimal-fleet>

References

- Liu, Y., et al. ProxRouter: Proximity-Weighted LLM Query Routing. *arXiv preprint arXiv:2510.09852*, 2024.
- Zhang, Y., et al. Beyond GPT-5: Making LLMs Cheaper and Better via Performance-Efficiency Optimized Routing. *arXiv preprint arXiv:2508.12631*, 2024.
- Chen, L., Zaharia, M., and Zou, J. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv preprint arXiv:2305.05176*, 2023.
- Zhang, H., et al. SHEPHERD: Serving DNNs in the Wild. *NSDI*, 2024.
- Hendrycks, D., et al. Measuring Massive Multitask Language Understanding. *ICLR*, 2021.